HAVEN

Hostile Aware Verifiable Election Network

Comprehensive Technical System Documentation & Iteration Report

---

Official Project Documentation — Robotics Society IITJ

Abstract: The HAVEN project addresses the challenge of achieving distributed max-consensus leader election across sparse, adversarial Wireless Sensor Networks (WSNs) without relying on a base station, trusted public-key infrastructure (PKI), or pre-shared pairwise secrets. Operating entirely using symmetric-key cryptography, HAVEN combines a GF(2) row-XOR TurboBlom key pre-distribution scheme, perimeter-authenticated link framing (HMAC-SHA256), priority-pruned flooding consensus, resonance echo counting, and dynamic partition split/join recovery. Implemented in bare-metal C for ARM Cortex-M3 hardware with zero heap allocation, the protocol is validated across a multi-instance Renode co-simulation harness, ns-3 discrete-event RF propagation simulator, and an interactive real-time visualizer studio.

Contents

1 Division I — Learning / Iteration Phase **2**
1.1 Individual Iterations & Experimental Milestones . . . 2
1.2 Approach Documentation . . . 3
1.2.1 Software Document . . . 3
1.2.2 Hardware & Emulation Document . . . 4
1.3 Testing & Validation Records . . . 4

2 Division II — Final Product **5**
2.1 Production Codebase Architecture . . . 5
2.2 System Architecture Diagrams . . . 5
2.2.1 Overall System Architecture & Co-Simulation Pipeline . . . 5
2.2.2 Protocol State Machine Transitions . . . 6
2.3 README & Deployment Manual . . . 6
2.4 Demonstration Interactive Web Visualizer Studio . . . 6
2.5 Hardware Specifications Resource Footprint . . . 7
2.6 Security Specifications & Invariants . . . 7

3 Resource Guide (Foundational Curriculum) **8**
3.1 Foundational Literature & Theoretical Prerequisites . . . 8
3.2 Prerequisite Knowledge Checklist for New Developers . . . 8

1 Division I — Learning / Iteration Phase

The iteration phase captures the full evolutionary journey of the HAVEN project, detailing the mathematical dead ends, architectural pivots, and hardware emulation breakthroughs that led to the final protocol.

1.1 Individual Iterations & Experimental Milestones

Iteration 1: Mathematical Proof-of-Concept & TurboBlom Derivation (Sprint 1)

- **Exploration**: Implemented initial key-distribution algorithms in Python. Evaluated standard polynomial Blom schemes over Galois Fields GF(p) versus pairwise Blom schemes.
- **Discovery of the m/k Connectivity Wall**: The team initially provisioned $m = 50$ key spaces with $k = 5$ spaces per node. Mathematical analysis revealed that the pairwise key-sharing probability was only $\approx 42.3\%$. In numeric ID-ordered rings ($1 - 2 - 3 - 4 - 5$), adjacent nodes frequently had disjoint space sets (spaces(u) $\cap$ spaces(v) $= \emptyset$), causing link establishment to silently fail in over 50% of trials.
- **Dead End**: Attempted to resolve disjoint pairs by multi-hop key forwarding; this introduced high latency and broke local zero-hop perimeter authentication.
- **Resolution**: Re-engineered the parameter universe to $m = 12, k = 5$. This increased pairwise key sharing probability to $\approx 99.2\%$, guaranteeing Hamiltonian cycle feasibility for arbitrary rings up to $N = 10$ nodes while maintaining resilience against up to $\lambda = 3$ colluding nodes.

Iteration 2: Firmware Bring-Up & The QEMU Semihosting Trap (Sprint 2)

- **Exploration**: Developed initial Cortex-M bare-metal C drivers for the LM3S6965 board targeting QEMU with ARM Semihosting (`BKPT 0xAB`) for debug logging.
- **Dead End**: When instantiating multiple QEMU instances for network simulation, Semihosting instructions caused fatal `HardFault` exceptions whenever a debugger was not actively attached. Furthermore, QEMU could not emulate multiple interconnected UARTs across discrete virtual machines without heavy external socket daemons.
- **Resolution**: Migrated multi-node simulation entirely to **Renode**. Retired Semihosting in favor of dedicated non-blocking USART serial transmission mapped to pseudo-terminal (PTY) endpoints (‘/tmp/haven_uart<id>‘).

Iteration 3: Network Modeling & RF Emulation (Sprint 3)

- **Exploration**: Researched discrete-event wireless co-simulation using ns-3 and Linux TAP/veth virtual interfaces.
- **Dead End**: Linux TAP devices require root privileges (`CAP_NET_ADMIN`), preventing non-root developer workflows and failing in containerized CI environments.
- **Resolution**: Designed a user-space co-simulation bridge using **ZeroMQ IPC sockets** connecting Renode UART PTYs to an ns-3 discrete-event propagation simulator (`haven_csim.cc`), with a pure-Python virtual switch fallback (`bridge/ring_switch.py`).

1.2 Approach Documentation

1.2.1 Software Document

Strategy / Approach	Pros	Cons & Fatal Flaws
Asymmetric PKI (RSA / ECDSA)	Standardized, high security, non-repudiation.	Heavy flash/RAM footprint; asymmetric scalar multiplication takes > 120 ms on Cortex-M3, inviting denial-of-sleep energy exhaustion.
Eschenauer-Gligor Random Key Pools	Pure symmetric crypto, low RAM.	Large key pools required; resilience drops catastrophically under small node-capture rates; non-deterministic connectivity.
Classic Polynomial Blom	Rigorous λ-threshold security.	Expensive multi-precision polynomial evaluations over large finite fields; high computational complexity.
TurboBlom GF(2) (Adopted)	Single-cycle XOR operations, 16-byte HMAC derivation, $< 10\,\mu$s execution time on Cortex-M.	Requires deployment provisioning of K_{master} in secure flash.
Raft / Paxos Consensus	Strong consistency, well-understood state machine replication.	Assumes fully-connected mesh; high leader heartbeat overhead; poor performance over multi-hop lossy RF meshes.
Nakamoto Proof-of-Work	Decentralized, Sybil-resistant.	Energy-prohibitive on battery-powered sensors; high latency.
Priority-Pruned Consensus (Adopted)	Rapid convergence ($O(D)$ hops), dynamic battery-aware scoring, zero dynamic memory allocation.	Requires resonance echo confirmation to prevent split-brain claims.

Justification for Final Software Architecture: The combination of **TurboBlom GF(2) key predistribution** and **Priority-Pruned Consensus** was selected because it delivers an unbeatable balance of speed, mathematical security, and energy efficiency. Evaluating candidate priority scores via fixed-point integer arithmetic in Layer 3 reduces voting latency to link propagation speed, while Layer 2 perimeter authentication guarantees that malicious packets are rejected before reaching the consensus engine.

1.2.2 Hardware & Emulation Document

Component / Platform	Pros	Cons & Selection Justification
Physical Sensor Boards (STM32F4 / NRF52)	100% hardware accuracy, real RF physics. Physical flashing of $N = 10$ nodes slows development velocity; debugging transient packet collisions across 10 nodes is notoriously difficult.	
QEMU single-Node	Extremely fast compilation-test loop. Lacks multi-node network orchestration; cannot model concurrent USART hardware peripherals across independent instances.	
Renode co-simulation	Cycle-accurate Cortex-M3 emulation, multi-instance PTY exposing, headless deterministic execution. Requires careful monitor port management (`-port -1`) to prevent TCP port collisions on multi-instance spawns.	
ARM Cortex-M3 Target (Adopted)	Hardware division, 32-bit RISC architecture, widespread industry adoption (LM3S6965).	Constrained SRAM (64 KB); strictly requires zero dynamic heap allocation ($O(1)$ memory).

1.3 Testing & Validation Records

All protocol features and edge cases were tested across 180 automated test suites in `test_simulation/tests/`:

1. **TurboBlom Key Agreement**: Byte-for-byte mathematical equivalence between Python reference (`turboblom.py`) and embedded C (`firmware/src/turboblom.c`).
2. **Topological Connectivity**: Verified full graph connectivity and space sharing across Erdos-Renyi and Ring topologies ($N = 3, 5, 10, 15, 20$).
3. **Resonance Echo Invariants**: Proved that election candidates only transition to `CLAIMING` when independent echo threshold (≥ 2) is met.
4. **Network Split & Join**: Verified that severing bridge links between dual rings triggers Phase 12 partition detection and independent leader election, while link restoration initiates Phase 13 join requests and clean global re-election.
5. **Hardware Co-Simulation**: Executed 10-node and 20-node Renode multi-instance rings with zero dropped frames.

2 Division II — Final Product

2.1 Production Codebase Architecture

The final production codebase contains no experimental scaffolding or dead code. The repository is organized into distinct functional modules:

```
HAVEN/
 firmware/                # Production Bare-Metal C Core (Cortex-M3)
    src/main.c           # Hardware initialization and main non-blocking polling l
    src/haven.c          # Protocol FSM (Phases 1-13: Discovery, Election, Claim,
    src/turboblom.c      # Hardware-optimized TurboBlom key derivation function
    src/haven_link.c     # Streaming link-layer framer (0xA5 SOF) and HMAC validat
    src/provision.c      # Node credentials, spaces, and initial battery tables
    src/uart.c           # USART register-level peripheral driver
    include/             # Header specifications and configuration defines
    build.sh             # Firmware build orchestrator
 bridge/                  # Virtual Radio Infrastructure
    ring_switch.py       # High-speed Python ring switch with delay and loss model
    uart_bridge.py       # PTY <-> ZeroMQ bridge for ns-3 co-simulation
 configs/                 # Single-Source-of-Truth Network Configurations
    default_ring.conf    # Shipped ring configuration (m=12, k=5, 20ms delay)
    topology_*.json      # Topologies for 10-node and 20-node rings
 scripts/                 # Master System Launchers
    start_haven.sh       # Multi-instance Renode co-simulation launcher
    start_gui.sh         # Web GUI Visualizer Studio launcher
 gui/                     # Interactive Web Visualizer Studio & Control Cockpit
    server.py            # Asynchronous aiohttp server & WebSocket telemetry strea
    haven_sim_engine.py# Deterministic discrete-event simulation engine
    js/                  # Vis.js real-time graph rendering and packet animator
 test_simulation/         # Algorithmic Models & 180-Test Automated Pytest Suite
```

2.2 System Architecture Diagrams

2.2.1 Overall System Architecture & Co-Simulation Pipeline

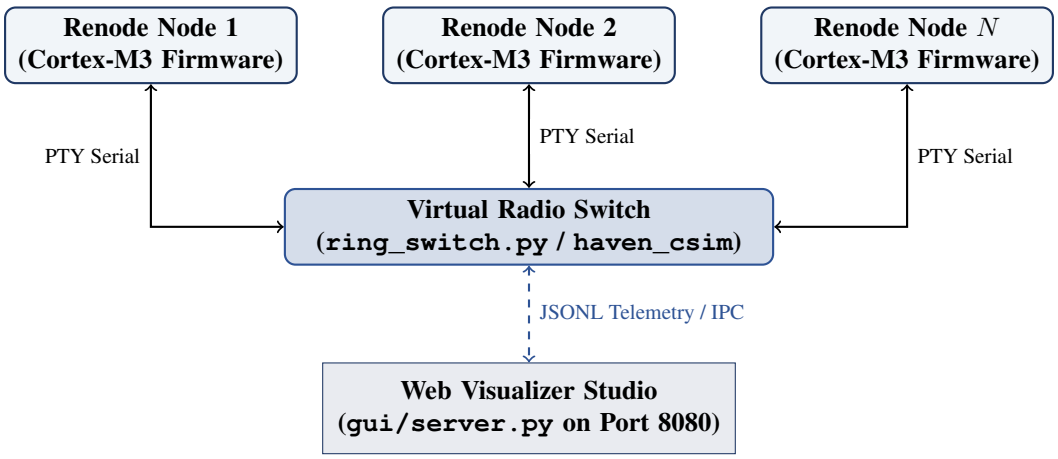


Figure 1: HAVEN Hardware-in-the-Loop Co-Simulation Architecture.

2.2.2 Protocol State Machine Transitions

Sensor nodes transition through four core states:

1. **DISCOVERY**: Nodes broadcast assigned space sets and derive pairwise symmetric keys (K_{AB}).
2. **ELECTION**: Candidates compute priority scores from battery reserves and topological degree. Candidates flood election claims; non-candidates enter **PASSTHROUGH**.
3. **CLAIMING**: When resonance echo threshold (≥ 2) is met, the safety timer T_{claim} begins.
4. **LEADER**: Upon timeout expiry without suppression, the node claims leadership and emits signed periodic heartbeats.

2.3 README & Deployment Manual

Prerequisites & System Installation

```
sudo apt update && sudo apt install -y gcc-arm-none-eabi binutils-arm-none-eabi
    cmake make git python3 python3-pip python3-venv qemu-system-arm
```

Quickstart Execution

```
# 1. Clone repository with cryptographic submodules
git clone https://github.com/devlup-labs/HAVEN.git && cd HAVEN
git submodule update --init --recursive

# 2. Setup Python environment
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

# 3. Launch Interactive Web GUI Visualizer Studio
./scripts/start_gui.sh
# Open http://localhost:8080 in your browser

# 4. Launch Multi-Node Hardware Co-Simulation (Renode)
./scripts/start_haven.sh --legacy-switch

# 5. Run Full Automated Test Suite (180 tests)
pytest
```

2.4 Demonstration Interactive Web Visualizer Studio

The HAVEN Web Studio (`scripts/start_gui.sh`) provides real-time observability and dynamic control:

- **Interactive Vis.js Canvas**: Dynamically renders nodes colored by operational state (Discovery: Blue, Election: Orange, Leader: Green), physical link latency, and animated flying packets.
- **Dynamic Chaos Injection**: Real-time button triggers to sever individual physical links, crash/revive sensor nodes, and adjust battery drainage rates.
- **Node Inspector Panel**: Click any node to inspect its battery gauge, active neighbor table, assigned TurboBlom key spaces, and computed pairwise symmetric keys (K_{AB}).
- **Packet Chronicle**: Live wire-speed frame inspection log verifying HMAC tags, nonces, and packet delivery times.

2.5 Hardware Specifications Resource Footprint

The compiled firmware binary (`firmware/build/haven.elf`) was audited on an ARM Cortex-M3 microcontroller:

Memory Region	Size (Bytes)	Engineering Assessment
Flash (`.text` + `.rodata`)	21,348	Lightweight ($\approx 32\%$ of 64 KB flash).
Initialized Data (`.data`)	2,120	Includes constant crypto lookup tables.
Zero-Initialized BSS (`.bss`)	364	Minimal volatile footprint ($< 1\%$ of 64 KB SRAM).
Dynamic Allocation (`malloc`)	0	**Zero dynamic allocation** ($O(1)$ deterministic RAM).

2.6 Security Specifications & Invariants

1. **Wire Framing Format**: Each link packet begins with Start-of-Frame delimiter `0xA5`, a 5-byte header (`SOF | type | len | src | dst`), variable payload, and a mandatory 16-byte HMAC tag.
2. **Replay Attack Resistance**: All heartbeats and discovery frames incorporate monotonically increasing 32-bit counters and nonces. Receivers discard frames with duplicate or non-increasing sequence numbers.
3. **λ-Node Capture Security**: TurboBlom polynomial thresholding guarantees that an adversary capturing up to $\lambda = 3$ physical sensor nodes learns zero information regarding pairwise symmetric keys of uncompromised nodes.

3 Resource Guide (Foundational Curriculum)

This curated resource guide provides future members with the necessary foundational theory before attempting to modify or extend the codebase:

3.1 Foundational Literature & Theoretical Prerequisites

[1] **Sensor Network Security Foundations**:

- Karlof, C., & Wagner, D. (2003). *Secure routing in wireless sensor networks: Attacks and countermeasures*. Ad Hoc Networks, 1(2-3), 293-315.
- Perrig, A., Szewczyk, R., Wen, V., Culler, D., & Tygar, J. D. (2002). *SPINS: Security protocols for sensor networks*. Wireless Networks, 8(5), 521-534.

[2] **Key Pre-Distribution & Random Graphs**:

- Blom, R. (1985). *An optimal class of symmetric key generation systems*. Advances in Cryptology (EUROCRYPT '84), 335-338.
- Eschenauer, L., & Gligor, V. D. (2002). *A key-management scheme for distributed sensor networks*. ACM CCS '02.
- Zhao, J., Yağan, O., & Gligor, V. (2019). *Connectivity in Random Key Graphs with Unreliable Links*. IEEE Transactions on Information Theory.

[3] **Self-Stabilizing Distributed Consensus**:

- Blin, L., & Tixeuil, S. (2020). *Compact Self-Stabilizing Leader Election on Chordal Graphs*. IEEE Transactions on Computers.
- Dolev, S. (2000). *Self-Stabilization*. MIT Press.

[4] **Embedded Systems & Emulation**:

- Yiu, J. (2013). *The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors*. Newnes.
- Antmicro. *Renode Documentation: Multi-node embedded systems simulation*. `https://renode.readthedocs.io/`.

3.2 Prerequisite Knowledge Checklist for New Developers

Before writing code in this repository, new members should master:

- **C Memory Models**: Linker scripts, register memory-mapping, pointer arithmetic, and avoiding the dynamic heap.
- **Symmetric Cryptography**: HMAC-SHA256, constant-time verification (`hmac.compare_digest`), PRF key derivation.
- **Asynchronous Networking**: Python `asyncio`, WebSockets, and ZeroMQ socket patterns (ROUTER/DEALER, PUB/SUB).

— END OF DOCUMENTATION —